

CI/CD con enfoque DevSecOps

Caso de estudio: Forseti

Facturación electrónica SRI (Ecuador) → pipeline completo del commit al cluster Kubernetes.

Hernán Jurado — Ingeniería en Software

UDLA · ISWZ3205 Procesos de Software

Julio 2026

Lámina 2 · El problema

Forseti procesa datos fiscales sensibles (RUCs, comprobantes electrónicos firmados con XAdES-BES).

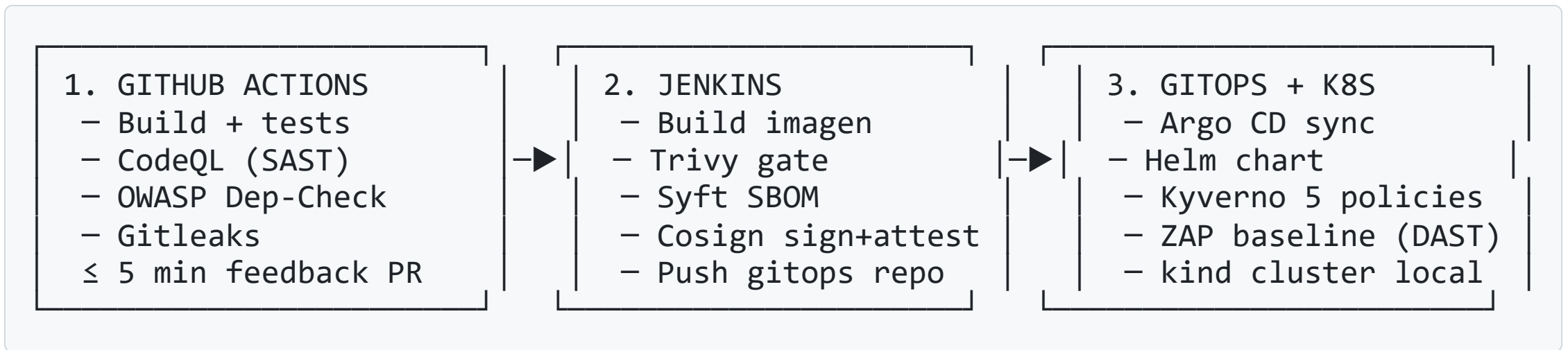
Riesgos concretos:

- **Compliance:** LOPDP Ecuador (2021) + reglamento SRI.
- **Cadena de suministro:** una lib npm/Maven comprometida = firmas SRI inválidas.
- **Secretos:** certificados `.p12` + master key AES-256-GCM en el flujo.

Pregunta guía: ¿cómo entregamos rápido *sin* aflojar la seguridad?

→ **Un pipeline CI/CD con enfoque DevSecOps.**

Lámina 3 · Arquitectura en 3 capas



Regla de oro: cada herramienta tiene un rol único. **Nada se pisa.**

Lámina 4 · Herramientas — matriz sin redundancia

#	Etap	Herramienta	Qué cubre (que las demás NO cubren)
1	CI orquestador	GitHub Actions	Feedback en el PR mismo
2	SAST	CodeQL	Bugs en código propio (Java + TS)
3	SCA (deps)	OWASP Dep-Check	CVEs de libs Maven/npm
4	Secretos	Gitleaks	Credenciales en historial Git
5	CD pesado	Jenkins	Release pipeline con etapas largas
6	Container scan	Trivy	CVEs en OS + libs de la imagen
7	SBOM	Syft	Inventario SPDX-JSON
8	Signing	Cosign	Firma keyless (Sigstore)
9	GitOps	Argo CD	Sync declarativo del cluster
10	Policy	Kyverno	Admission control YAML-nativo
11	DAST	OWASP ZAP	Vulns en runtime del deploy

Lámina 5 · GitHub Actions — CI que no bloquea

```
jobs:
  backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: '21' }
      - run: mvn -B verify    # incluye ArchUnit + JUnit
```

5 workflows en paralelo:

-  ci.yml — build + test
-  codeql.yml — SAST
-  dependency-check.yml — SCA
-  gitleaks.yml — secretos
-  docker-build.yml — Buildx + Trivy + Syft + Cosign keyless (solo main)

Lámina 6 · Jenkins — release engineering con gates

Pipeline declarativo (`Jenkinsfile`) con 9 stages.

```
stage('Trivy – Gate CRITICAL') {
  steps {
    sh 'trivy image --severity CRITICAL --exit-code 1 --ignore-unfixed ${IMG}'
    // ↑ si hay CVE CRITICAL fixable, rompe el build
  }
}

stage('Cosign – firma + attest SBOM') {
  steps {
    sh 'cosign sign --key $KEY ${IMG}'
    sh 'cosign attest --predicate sbom.spdx.json --type spdxjson ${IMG}'
  }
}
```

Diferencia clave con GH Actions: llave propia (no keyless) + gate estricto + DAST + push a repo GitOps.

Lámina 7 · GitOps con Argo CD — el cluster es lo que dice Git

Regla: el cluster **no acepta** cambios directos. Todo viene de un commit al repo

forseti-devsecops-gitops .

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata: { name: forseti-staging }
spec:
  source:
    repoURL: https://github.com/TARIUS-S-A-S/forseti-devsecops-gitops
    path: environments/staging
    helm: { valueFiles: [values.yaml] }
  syncPolicy:
    automated: { prune: true, selfHeal: true }
```

Beneficios:

- Auditoría automática (git log = historia de deploys)

- Rollback = git revert

Lámina 8 · Kyverno — la última línea de defensa

5 políticas en modo **Enforce** en el cluster:

Policy	Rechaza si...
verify-forseti-image-signatures	Imagen sin firma Cosign válida
require-resource-limits	Container sin requests o limits
disallow-latest-tag	Tag :latest en forseti-prod
disallow-privileged-containers	privileged=true , hostNetwork, hostPID, hostIPC
require-run-as-non-root	runAsNonRoot no está en true

Ejemplo de bloqueo real:

```
$ kubectl -n forseti-prod run test --image=nginx:latest
Error: policy disallow-latest-tag/block-latest: en forseti-prod está prohibido
el tag :latest o imagen sin tag. Usá sha-... o vX.Y.Z.
```


Lámina 9 · DevSecOps en números — evidencias

Métrica	Resultado
Tiempo commit → pod running en staging	≈ 22 min
Tiempo hasta feedback en PR	≤ 5 min
Herramientas de seguridad por release	7
CVEs CRITICAL fixables no mitigados	0
Secretos en historial	0
Pods sin firma Cosign en cluster	0 (Kyverno los rechaza)
CodeQL alerts open	0

Defensa en profundidad: ninguna herramienta hace redundante a las demás.
SAST atrapa lo del código, SCA lo de las deps, DAST lo del runtime, signing lo del supply chain, policy lo del cluster.

Lámina 10 · Aprendizajes + próximos pasos

Lo que funcionó:

- Separar CI rápido (Actions) de CD pesado (Jenkins) — cada uno hace lo que hace mejor
- GitOps + Kyverno — la firma Cosign deja de ser decorativa
- Cosign keyless (OIDC) — supply chain security *sin* HSM

Lo que quedó pendiente:

1. Renovate/Dependabot autónomo con auto-merge por tests
2. Runtime security con Falco (complementa DAST post-mortem)
3. Sealed-Secrets / External-Secrets-Operator (hoy son placeholders)
4. Argo Rollouts para canaries y blue-green